

# JAVA PROGRAMMING LABORATORY

Experiment No.: 06

Date: 23/09/26

## Title: Demonstration of TreeMap Methods in Java

### Objective / Aim

To write and execute a Java program to demonstrate commonly used methods of the TreeMap class, including insertion, retrieval, searching, key navigation, entry retrieval, and removal of key-value pairs.

### System Requirements

#### Hardware Requirements

| Component        | Requirement                       |
|------------------|-----------------------------------|
| Processor        | Intel/AMD processor or equivalent |
| RAM              | Minimum 2 GB                      |
| Storage          | At least 100 MB free space        |
| Keyboard/Monitor | Standard keyboard and display     |

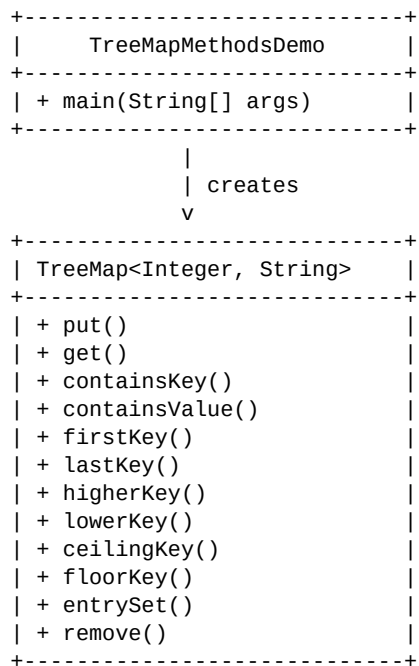
#### Software Requirements

| Software         | Requirement                                               |
|------------------|-----------------------------------------------------------|
| Operating System | Windows / Linux / macOS                                   |
| Java             | JDK 8 or later                                            |
| Editor           | Notepad, VS Code, Eclipse, IntelliJ IDEA, or any Java IDE |
| Execution        | Command Prompt / Terminal or IDE                          |

### Algorithm / Logic Description

1. Import the TreeMap class from java.util.
2. Create a TreeMap with Integer keys and String values.
3. Insert five student records using put().
4. Display the TreeMap. TreeMap automatically stores entries in ascending order of keys.
5. Retrieve the student name for key 102 using get().
6. Check whether key 103 and value "Rahul" exist using containsKey() and containsValue().
7. Display the smallest and largest keys using firstKey() and lastKey().
8. Find keys immediately higher and lower than 102 using higherKey() and lowerKey().
9. Find the nearest key greater than or equal to 103 and less than or equal to 103 using ceilingKey() and floorKey().
10. Display all key-value pairs using entrySet().
11. Remove the entry with key 104 using remove() and display the updated TreeMap.
12. Compile and execute the program and verify the output.

## UML Class Diagram



## Source Code

```
import java.util.TreeMap;

class TreeMapMethodsDemo {

    public static void main(String[] args) {

        // Creating a TreeMap
        TreeMap<Integer, String> students = new TreeMap<>();

        // put(K key, V value)
        students.put(103, "Arjun");
        students.put(101, "Rahul");
        students.put(105, "Sneha");
        students.put(102, "Priya");
        students.put(104, "Kiran");

        System.out.println("After put(): " + students);

        // get(Object key)
        System.out.println("Student with ID 102: "
            + students.get(102));

        // containsKey(Object key)
        System.out.println("Contains key 103? "
            + students.containsKey(103));

        // containsValue(Object value)
        System.out.println("Contains value Rahul? "
            + students.containsValue("Rahul"));

        // firstKey()
        System.out.println("First key: " + students.firstKey());

        // lastKey()
        System.out.println("Last key: " + students.lastKey());

        // higherKey(K key)
        System.out.println("Key higher than 102: "
            + students.higherKey(102));

        // lowerKey(K key)
        System.out.println("Key lower than 102: "
            + students.lowerKey(102));

        // ceilingKey(K key)
        System.out.println("Ceiling key of 103: "
            + students.ceilingKey(103));

        // floorKey(K key)
        System.out.println("Floor key of 103: "
            + students.floorKey(103));

        // entrySet()
        System.out.println("Key-Value pairs: "
            + students.entrySet());

        // remove(Object key)
        students.remove(104);
        System.out.println("After remove(104): "
            + students);
    }
}
```

## Completion and Execution Commands

1. Save the program with the filename **TreeMapMethodsDemo.java**.
2. Open Command Prompt or Terminal in the folder containing the file.

3. Compile the program:

```
javac TreeMapMethodsDemo.java
```

4. Execute the program:

```
java TreeMapMethodsDemo
```

## Output

```
After put(): {101=Rahul, 102=Priya, 103=Arjun, 104=Kiran, 105=Sneha}
Student with ID 102: Priya
Contains key 103? true
Contains value Rahul? true
First key: 101
Last key: 105
Key higher than 102: 103
Key lower than 102: 101
Ceiling key of 103: 103
Floor key of 103: 103
Key-Value pairs: [101=Rahul, 102=Priya, 103=Arjun, 104=Kiran, 105=Sneha]
After remove(104): {101=Rahul, 102=Priya, 103=Arjun, 105=Sneha}
```

## Result / Completion

The Java program was successfully compiled and executed. The commonly used TreeMap methods were demonstrated, and the entries were maintained in ascending order of their keys.